

Rethinking How We Store a Distilled Dataset: SPEED

NeurIPS 2023 · A sparse, shared parameterization for epitomic dataset distillation that sets state-of-the-art at a fraction of the storage

Modern deep learning runs on enormous datasets, and those datasets are expensive to store, move, and train on. Dataset distillation offers an appealing shortcut: compress a large dataset into a tiny synthetic one that still trains a model to competitive accuracy. The catch is that the synthetic budget is minuscule, sometimes as little as a single image per class, so every parameter spent on the synthetic set has to earn its place.

Most work in this area has poured its energy into the matching objective, the loss that aligns the synthetic data with the real data through gradients, feature distributions, or training trajectories. How the synthetic images are actually parameterized has been an afterthought: each image is usually optimized on its own, as an independent block of pixels or features. That naive scheme quietly ignores a fact every computer-vision researcher knows well, which is that natural images are full of repetition. Textures recur, patches echo one another, and similar structures show up across many images and many classes.

SPEED, presented at NeurIPS 2023, takes that redundancy seriously. Short for Sparse Parameterization for Epitomic Dataset Distillation, it reframes the synthetic dataset as a shared dictionary plus a set of sparse per-image codes, borrowing the classical idea behind dictionary learning and sparse coding. Concretely, it introduces three pieces: Spatial-Agnostic Epitomic Tokens (SAETs), a small pool of tokens reused by every image patch regardless of where it sits; Sparse Coding Matrices (SCMs), which select only the most significant tokens for each image; and a Feature-Recurrent Network (FReeNet) that reassembles those tokens back into full, high-resolution synthetic images, patch by patch.

The framework at a glance

The overview below traces one synthetic image through the pipeline. On the left, a shared bank of SAETs, drawn as H parallel heads, is combined with a per-image SCM in which only a handful of entries are non-zero. Those sparse combinations feed FReeNet, a compact transformer-style block with Add & Norm and MLP layers that is applied recurrently, reusing the same shared tokens and codes at every step to build hierarchical image patches. The synthesized images on the right are what the matching objective actually compares against real data.

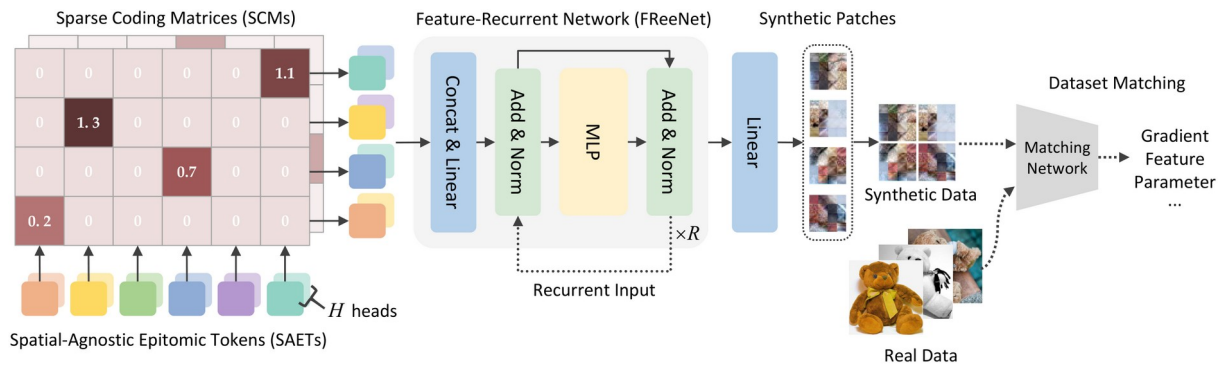


Figure 1. The SPEED framework: a shared dictionary of Spatial-Agnostic Epitomic Tokens and sparse per-image coding matrices are recurrently assembled by FReeNet into synthetic images, which are then matched against real data.

Paper:

https://proceedings.neurips.cc/paper_files/paper/2023/hash/9e8889198d16fb79926e71adbe38cae4-Abstract-Conference.html Code: <https://github.com/MIV-XJTU/SPEED>

The problem: redundancy hides in the parameterization

To see why parameterization matters, picture the tightest setting, one image per class on CIFAR100. A naive method spends the entire budget drawing exactly one synthetic image per class, pixel by pixel, with no sharing between classes. Yet many of those pixels encode the same edges, textures, and color blobs over and over. A few recent methods began to exploit relationships between synthetic images, but none removed redundancy in a spatially-agnostic way, meaning independent of where a feature happens to appear. SPEED's bet is that if you spend almost none of the budget on a shared dictionary and per-image sparse codes, and let a small network reconstruct the rest, you can fit far more informative content into the same storage.

How SPEED parameterizes a dataset

Each synthetic image is written as a function of two things: the shared dictionary of SAETs, which is the same for the entire dataset, and a per-image Sparse Coding Matrix that selects and weights a few tokens. Multi-head sparse coding combines them, so a single image is expressed as a sparse mixture of shared building blocks rather than an isolated grid of pixels. Because the dictionary and the network are shared across every image, the only thing that has to be stored per image is its short list of non-zero code entries.

FReeNet then turns those sparse features into an image. It is a compact, transformer-style network, an MLP with residual connections and layer normalization, applied recurrently over R blocks so that a small set of weights can synthesize hierarchical, high-resolution patches. Training minimizes the chosen matching objective together with an L1 sparsity penalty that pushes most code entries toward zero. After training, each coding matrix is pruned to its top- k non-zero entries and stored in a sparse format, so the coding cost scales with only those few entries rather than the full dictionary.

The payoff is concrete. Under the same CIFAR100 budget that a naive method spends on a single image per class, SPEED's shared dictionary and sparse codes leave room for roughly 11 synthetic images per class. More images, all drawn from a shared and reusable representation, mean more of the information a downstream model needs, packed into the same tiny budget.

Results: state-of-the-art at a fraction of the storage

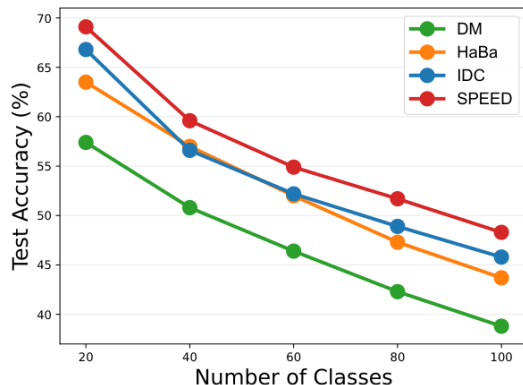
SPEED is evaluated on three standard benchmarks, CIFAR10 and CIFAR100 at 32×32 and TinyImageNet at 64×64, plus six high-resolution ImageNet subsets at 128×128 (ImageNette, ImageWoof, ImageFruit, ImageMeow, ImageSquawk, and ImageYellow), each with 10 classes. Everything is compared under equal storage budgets, counted in parameters per class at IPC 1, 10, and 50. Across all of these, SPEED sets a new state-of-the-art (SOTA).

Table 1. SPEED's headline results under tight storage budgets.

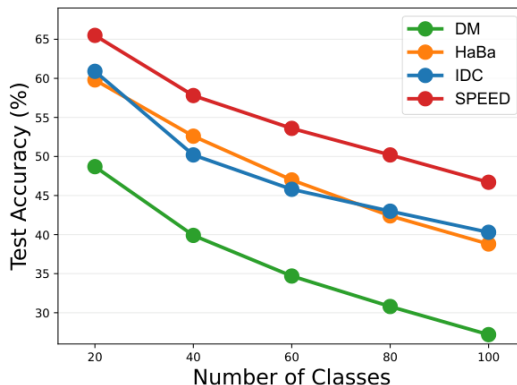
Benchmark	Setting	SPEED Acc.	vs. prior SOTA
CIFAR100	IPC 1	40.0%	+6.0%
TinyImageNet	IPC 1	26.9%	+10.9%
ImageNet subsets (avg)	IPC 1	—	+11.2%
ImageSquawk	IPC 10	71.8%	+15.0%

The headline numbers are worth reading twice. At the tightest IPC 1 budget, SPEED reaches 40.0% on CIFAR100, a +6.0% gain over the previous best, and 26.9% on TinyImageNet, a +10.9% gain. Across the ImageNet subsets it improves on the prior SOTA by +11.2% on average at the same budget. Most striking, its IPC 1 results rival what earlier methods achieved at IPC 10 while using only about 10% of their parameters. On ImageSquawk at IPC 10 it reaches 71.8%, a +15.0% jump. The synthetic data also transfers to unseen backbones, reaching 51.5% on a ViT for CIFAR10 IPC 10, a +3.6% gain.

Those gains hold up under stress. In a continual-learning setup, where classes arrive in steps, SPEED stays on top at every stage. The plots below track test accuracy as the number of classes grows, on both a ConvNet and a ResNet18; the SPEED curve sits above DM, HaBa, and IDC throughout.



(a) ConvNet



(b) ResNet18

Figure 2. Continual learning: SPEED keeps the best test accuracy at every step, on both ConvNet and ResNet18.

Robustness tells the same story. Training a model on SPEED's synthetic data and testing under corruption, across 14 corruption types at 5 severity levels, the authors compare accuracy. On ResNet18, SPEED nearly doubles the accuracy of prior methods under essentially every corruption.

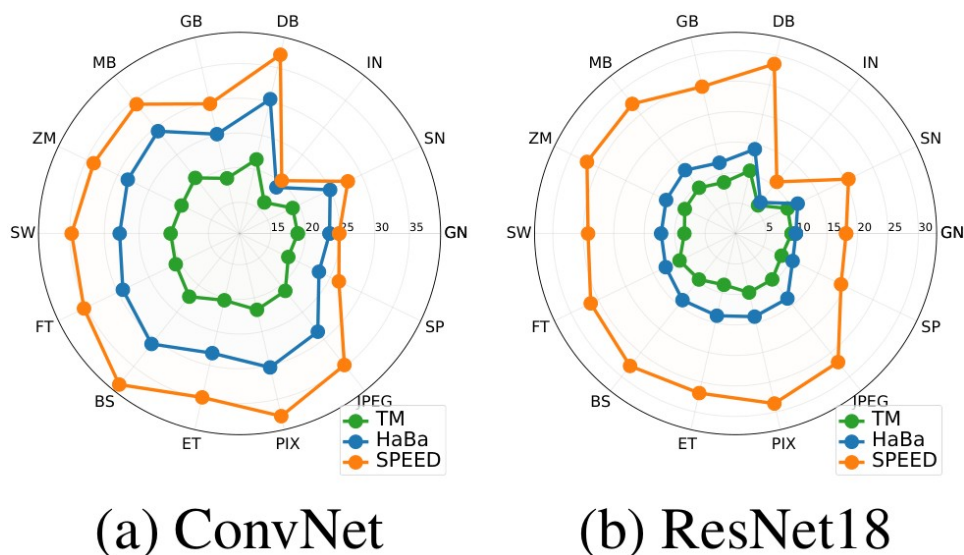


Figure 3. Robustness across many corruption types: SPEED (orange) clearly outperforms TM and HaBa on both ConvNet and ResNet18.

Sparsification is nearly free

A natural worry is that aggressively pruning the codes would gut the synthetic data. SPEED's ablations say otherwise. Pruning each coding matrix to just $k=48$ non-zero entries, about 307K parameters, meets the storage budget while keeping ConvNet accuracy at 73.5%, barely below the 74.0% of the full 15M-parameter model. Push the pruning too far, down to $k=12$ and about 106K parameters, and accuracy does collapse, to 57.8%, so a moderate k is the sweet spot, and it even helps generalization to unseen architectures. On the network side, 2 recurrent blocks ($R=2$) and 3 heads ($H=3$) work best.

The qualitative check is the most convincing. The grids below show synthetic ImageSquawk samples before and after sparsification down to 0.3% density. The two are nearly indistinguishable: pruning removes almost all of the stored values while preserving the global semantics of each image.

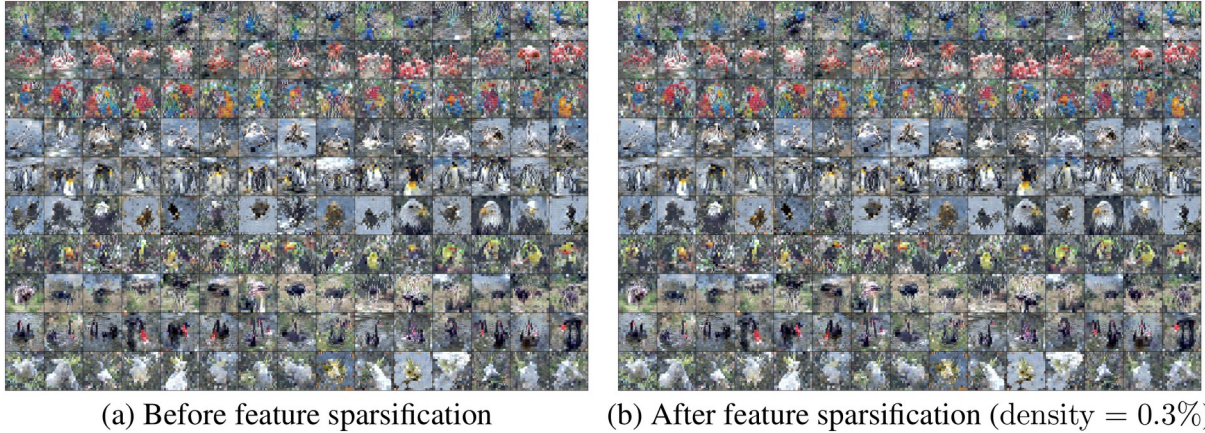


Figure 4. ImageSquawk synthetic samples before and after feature sparsification to 0.3% density: the pruned images look almost identical to the originals.

Why it matters

The lasting message of SPEED is that how you parameterize a synthetic dataset deserves as much attention as the matching objective you optimize. By treating the dataset as a shared dictionary of spatial-agnostic tokens, sparse per-image codes, and a small recurrent network that reassembles them, SPEED removes the spatial redundancy that naive methods leave on the table. The result is state-of-the-art distillation at a fraction of the storage, with the biggest gains on high-resolution data, plus better transfer to unseen architectures and stronger robustness to corruption. It is also a drop-in idea: because it is a parameterization rather than a new objective, it plugs into gradient, distribution, and trajectory matching alike. Sparse, shared representation, in other words, is a general lever for making tiny synthetic datasets do far more.

None of this comes entirely for free. SPEED adds a small dictionary and a recurrent synthesizer to the distillation loop, and the sparsity level k has to be chosen with some care, since pruning too aggressively does cost accuracy. But the framework asks for no change to the matching objective, and it leaves the door open to richer dictionaries, learned sparsity patterns, and larger-scale datasets. For anyone building compact synthetic data, the practical lesson is simple: treat parameterization as a first-class design choice, not an afterthought to bolt on after the objective is already set.